INSTITUTO POLITÉCNICO NACIONAL · ESCUELA SUPERIOR DE CÓMPUTO (ESCOM)
La Técnica al Servicio de la Patria · Subdirección Académica

PRÁCTICA INTEGRADORA 1ER PARCIAL · DOCUMENTO TÉCNICO OFICIAL

PetMatch Prolog: Sistema Experto Basado en Programación Lógica para la Elección y Adopción Responsable de Mascotas

Desarrollo de Base de Conocimientos en SWI-Prolog, Interfaz Gráfica Web, Contenerización con Docker (Alternativa B) y Despliegue en Internet.



Figura 1. Representación visual del dominio del sistema experto: evaluación lógica de compatibilidad hogar-mascota.

Campo Institucional	Información del Proyecto
Unidad de Aprendizaje	Paradigmas de Programación (Paradigma Lógico · Lenguaje: SWI-Prolog)
Nombre del Sistema Experto	PetMatch Prolog — Asistente para la Elección y Adopción de Mascotas
Ruta de Despliegue Elegida	Alternativa B: Aplicación Contenerizada con Docker (`swipl:stable`) y GitHub
Integrantes del Equipo	**Diego Perea Medina** · **Cortés Cruz Gerardo**
Grupo y Fecha de Entrega	Grupo: _____ · Fecha de entrega: Octubre de 2026

2. Introducción

Los sistemas expertos constituyen una de las aplicaciones más representativas de la Inteligencia Artificial simbólica y de la programación declarativa. A diferencia de los programas imperativos convencionales —donde el flujo de control y las estructuras de datos se encuentran estrechamente acoplados mediante secuencias de condicionales anidados—, un sistema experto separa de manera explícita la **Base de Conocimientos** (hechos y reglas del dominio) del **Motor de Inferencia** encargado de razonar sobre dicha información mediante unificación y resolución lógica.

En este documento técnico se presenta el diseño, implementación, integración web, contenerización con Docker y validación en producción de **PetMatch Prolog**, un sistema experto desarrollado en **SWI-Prolog** cuya finalidad es orientar a familias y adoptantes en la elección responsable de perros y gatos (tanto de raza como perfiles de refugio). El proyecto abarca todas las etapas de ingeniería de software solicitadas en la práctica integradora: desde el modelado formal de 61 hechos y 15 reglas lógicas no triviales en el archivo `mascotas_kb.pl`, hasta la exposición de un servidor HTTP nativo en Prolog (`server.pl`), una interfaz gráfica accesible desde cualquier navegador web y su empaquetado reproducible en un contenedor Docker publicado en Internet.

3. Planteamiento del Problema (Etapa 1: Definición)

En la vida cotidiana, la elección de un animal de compañía suele realizarse por impulso estético o desconocimiento de las necesidades biológicas y conductuales de cada especie o raza. De acuerdo con estadísticas de refugios animales, una proporción significativa de los abandonos y devoluciones ocurre durante los primeros seis meses posteriores a la adopción debido a incompatibilidades críticas entre el estilo de vida del adoptante y los requerimientos del animal: perros de pastoreo o alta energía confinados en departamentos pequeños sin paseos suficientes, razas no hipoalergénicas introducidas en hogares con alergias respiratorias, animales de baja tolerancia infantil en familias con niños pequeños, o costos veterinarios y de estética que superan el presupuesto mensual familiar.

- **Utilidad en la vida cotidiana:** Prevenir adopciones fallidas y fomentar la tenencia responsable proporcionando un diagnóstico objetivo, auditable y sustentado en reglas lógicas antes de llevar una mascota a casa.
- **Perfil de usuarios:** Personas individuales, estudiantes, parejas, familias con niños y adoptantes primerizos o experimentados que desean conocer qué razas de perros, razas de gatos o perfiles de animales rescatados de refugio se adaptan genuinamente a su entorno.
- **Información de entrada recibida por el sistema:** Siete variables del estilo de vida del usuario: (1) tipo de vivienda disponible, (2) tiempo libre diario para paseos y atención, (3) presencia de alergias en casa, (4) presencia y edad de niños en el hogar, (5) nivel de experiencia previa con mascotas, (6) presupuesto mensual disponible y (7) preferencia de especie.
- **Alcances y limitaciones:** El sistema evalúa 24 perfiles representativos (9 perros de raza, 7 gatos de raza y 8 perfiles de refugio), infiere múltiples recomendaciones ordenadas por porcentaje de afinidad y diagnostica causas de descarte. Su alcance es orientativo y preventivo; no sustituye la valoración médica veterinaria ni alergológica clínica.

4. Objetivo General y Objetivos Específicos

4.1. Objetivo General

Diseñar, implementar, contenerizar y desplegar en Internet una aplicación web interactiva cuyo motor de razonamiento esté íntegramente programado en SWI-Prolog (`mascotas_kb.pl` y `server.pl`), capaz de inferir recomendaciones de adopción de mascotas mediante unificación, recursividad y retroceso (backtracking), utilizando Git, GitHub y Docker.

4.2. Objetivos Específicos

1. Modelar una base de conocimientos coherente en Prolog con más de 20 hechos significativos sobre razas caninas, felinas y animales de refugio, así como jerarquías cuantitativas de espacio, tiempo, experiencia y presupuesto.
2. Construir al menos 10 reglas lógicas no triviales y 5 predicados propios diferenciados que implementen restricciones de exclusión estricta (alergias y seguridad infantil), compatibilidad relacional y cálculo recursivo de afinidad sobre listas.
3. Aprovechar el mecanismo de retroceso (backtracking) y el meta-predicado `findall/3` para obtener múltiples soluciones ordenadas y explicar las reglas aplicadas en cada conclusión.
4. Desarrollar una interfaz gráfica web responsiva que se comunique mediante peticiones HTTP y JSON con el motor SWI-Prolog sin requerir que el usuario conozca la sintaxis lógica.
5. Empaquetar el sistema completo en una imagen Docker basada en `swipl:stable`, versionar el código en GitHub mediante commits descriptivos y validar los escenarios de prueba en producción.

5. Justificación del Uso de Prolog y del Paradigma Lógico

El problema de selección y adopción de mascotas es intrínsecamente un problema de **satisfacción de restricciones multivariables y deducción basada en reglas**. Modelar este dominio en un lenguaje imperativo tradicional requeriría múltiples bucles y decenas de bloques condicionales `if/else` acoplados que dificultan agregar nuevas reglas o explicar por qué se tomó una decisión. En contraste, Prolog resulta ideal por cuatro razones técnicas fundamentales:

- **Representación declarativa del conocimiento:** Cada mascota y sus requerimientos se expresan como un hecho atómico autocontenido `mascota/12`, separado completamente de la lógica de evaluación.

- **Unificación bidireccional y polimorfismo declarativo:** El predicado `compatible_especie/2` permite que una misma regla filtre una especie específica (`perro` o `gato`) o unifique libremente con cualquier especie cuando el usuario elige `cualquiera`, sin duplicar código.
- **Búsqueda exhaustiva mediante Backtracking:** Cuando existen varias mascotas compatibles con el hogar del usuario, el mecanismo de resolución SLD con retroceso explora automáticamente todo el árbol de búsqueda y recupera todas las soluciones válidas.
- **Explicabilidad y trazabilidad lógica:** Prolog permite construir términos compuestos que registran exactamente qué reglas se cumplieron (`explicacion_compatibilidad/8`) y qué reglas fallaron en los candidatos descartados (`diagnosticar_descarte/9`).

6. Descripción de la Base de Conocimientos (Etapa 2: Diseño)

El archivo `prolog/mascotas_kb.pl` supera ampliamente todos los requisitos mínimos establecidos en la sección 4.1 de la práctica, integrando un total de **61 hechos significativos**, **15 reglas lógicas** y **14 predicados propios exportados**:

Elemento Lógico	Mínimo Exigido	Implementado	Detalle en `prolog/mascotas_kb.pl`
Hechos significativos	20 hechos	**61 hechos**	24 hechos `mascota/12` + 24 hechos `cuidados_especificos/2` + 13 hechos `rango_*/2`.
Reglas no triviales	10 reglas	**15 reglas**	Reglas de espacio, tiempo, alergias, niños, experiencia, presupuesto, recursión y diagnóstico.
Predicados propios	5 predicados	**14 predicados**	Predicados modulares claramente diferenciados por aridad y responsabilidad lógica.
Inferencias distintas	3 conclusiones	**4 conclusiones**	`compatibilidad_optima_ideal`, `compatibilidad_alta_recomendada`, `compatibilidad_adecuada_viable` y descarte.
Casos de prueba	10 casos	**12 casos**	Documentados en `tests/pruebas_consola.pl` y verificables en un clic desde la interfaz web.

6.1. Estructura de los Hechos Principales

Cada perfil animal está modelado mediante el predicado de aridad 12 `mascota(ID, Nombre, Especie, CategoriaOrigen, Tamano, Energia, EspacioMinimo, TiempoMinimo, Hipoalergenico, ToleranciaNinos, ExperienciaRequerida, PresupuestoMensual)`, acompañado por su lista de cuidados en `cuidados_especificos(ID, ListaCuidados)` y los hechos relacionales de orden `rango_espacio/2`, `rango_tiempo/2`, `rango_experiencia/2` y `rango_presupuesto/2`:

```
% Ejemplos de hechos significativos en prolog/mascotas_kb.pl:
mascota(schnauzer_mini, 'Schnauzer Miniatura', perro, raza, pequeno, media,
        departamento_pequeno, moderado, si, alta, principiante, moderado).

mascota(golden_retriever, 'Golden Retriever', perro, raza, grande, alta,
        casa_con_patio, alto, no, alta, principiante, alto).

mascota(gato_azul_ruso, 'Gato Azul Ruso', gato, raza, pequeno, baja,
        departamento_pequeno, poco, si, media, principiante, moderado).

mascota(mestizo_hipoalergenico, 'Cruza de Poodle Rescatado (Refugio)', perro, refugio, pequeno, media,
        departamento_pequeno, moderado, si, alta, principiante, economico).

cuidados_especificos(schnauzer_mini, [cepillado_barba_y_pelo, corte_peluqueria_bimestral, paseos_con_estimulacion_mental]).
```

7. Explicación de las Reglas y Predicados Más Importantes (Etapa 3)

El motor de inferencia articula cuatro familias de reglas lógicas dentro de `prolog/mascotas_kb.pl`:

7.1. Reglas Relacionales de Jerarquía (Espacio, Tiempo, Experiencia y Presupuesto)

En lugar de comparar únicamente igualdad exacta (lo cual impediría que una persona con una casa con jardín grande adopte un perro que solo requiere un departamento pequeño), las reglas mapean cada átomo cualitativo a su nivel numérico y verifican que la capacidad del adoptante sea mayor o igual a la exigencia mínima del animal:

```
compatible_espacio(ViviendaUsuario, EspacioMinimoMascota) :-
    rango_espacio(ViviendaUsuario, NivelUsuario),
    rango_espacio(EspacioMinimoMascota, NivelMascota),
    NivelUsuario >= NivelMascota.

compatible_tiempo(TiempoUsuario, TiempoMinimoMascota) :-
    rango_tiempo(TiempoUsuario, NivelUsuario),
    rango_tiempo(TiempoMinimoMascota, NivelMascota),
    NivelUsuario >= NivelMascota.
```

7.2. Reglas de Exclusión Estricta por Salud y Seguridad Infantil

Utilizan unificación directa en la cabeza de las cláusulas (con la variable anónima _ cuando no existe restricción) para garantizar que un hogar con alergias solo reciba candidatos hipoalergénicos (`si`) y que un hogar con niños pequeños menores de 6 años (`pequenos`) exija exclusivamente animales con tolerancia infantil `alta`:

```
seguro_alergias(no, _).
seguro_alergias(si, si).

seguro_ninos(ninguno, _).
seguro_ninos(pequenos, alta).
seguro_ninos(mayores, alta).
seguro_ninos(mayores, media).
```

7.3. Reglas Recursivas sobre Listas (Cálculo de Afinidad y Conteo de Cuidados)

Para cumplir con el objetivo específico de aplicar **recursividad en Prolog**, se implementaron los predicados recursivos `contar_elementos/2` (que recorre la lista de cuidados de cada mascota) y `sumar_factores/2` (que suma recursivamente los pares `Factor-Puntos` derivados de la holgura de espacio, tiempo, experiencia, presupuesto y bonificación por adopción en refugio):

```
contar_elementos([], 0).
contar_elementos([_|Cola], Total) :-
    contar_elementos(Cola, SubTotal),
    Total is SubTotal + 1.

sumar_factores([], 0).
sumar_factores([_-Puntos|Cola], Total) :-
    sumar_factores(Cola, SubTotal),
    Total is SubTotal + Puntos.

calcular_puntaje_afinidad(FactoresBonificacion, PuntajeBase, PuntajeFinal) :-
    sumar_factores(FactoresBonificacion, Bono),
    PuntajeSinTope is PuntajeBase + Bono,
    (PuntajeSinTope > 100 -> PuntajeFinal = 100 ; PuntajeFinal = PuntajeSinTope).
```

7.4. Regla Principal de Inferencia y Recolección con Backtracking

El predicado `recomendar_mascota/12` conjuga todas las reglas anteriores, infiere una de las tres categorías de recomendación mediante `clasificar_afinidad/2` (`compatibilidad_optima_ideal` para ≥90 puntos, `compatibilidad_alta_recomendada` para 80–89 puntos y `compatibilidad_adecuada_viable` para 70–79 puntos) y construye la trazabilidad de reglas en `explicacion_compatibilidad/8`. Por su parte, `obtener_recomendaciones/8` emplea `findall/3` para forzar el backtracking completo y ordenar las soluciones de mayor a menor afinidad.

8. Diagrama de Arquitectura del Sistema (Etapa 5)

Siguiendo la **Alternativa B (Aplicación contenerizada con Docker)** combinada con las bibliotecas HTTP nativas de SWI-Prolog, el sistema mantiene una arquitectura limpia sin lenguajes intermediarios innecesarios. Todo el razonamiento ocurre verdaderamente dentro del intérprete de SWI-Prolog:

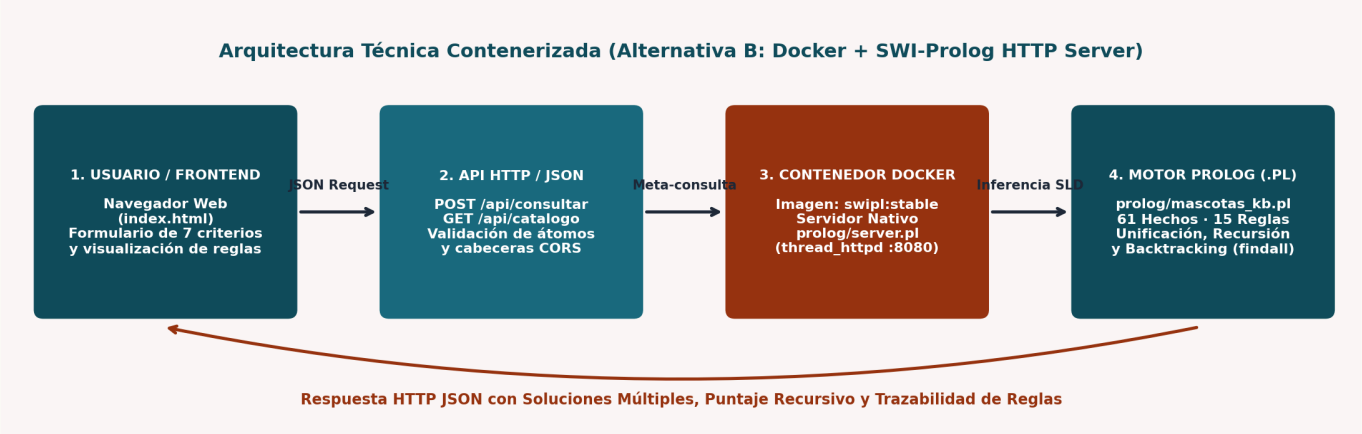


Figura 2. Flujo de comunicación cliente-servidor entre la interfaz web, la API HTTP JSON y el motor SWI-Prolog dentro del contenedor Docker.

9. Descripción del Funcionamiento de la Interfaz Gráfica (Etapa 4)

La interfaz gráfica (`frontend/index.html`) fue diseñada pensando en cualquier usuario que desconozca la sintaxis de Prolog, organizando la experiencia en cuatro pestañas interactivas:

- **Pestaña 1 — Asistente de Adopción (Inferencia):** Presenta un formulario claro con los 7 parámetros de entrada, botones de escenarios rápidos, un visor en tiempo real de la consulta Prolog generada (`?- obtener_recomendaciones(...).`), tarjetas detalladas para cada solución obtenida por backtracking con su porcentaje de afinidad, lista de cuidados específicos y el desglose exacto de las 7 reglas lógicas satisfechas, además de un panel desplegable que ejecuta `diagnosticar_descarte/9` para explicar por qué fueron descartadas las demás mascotas.
- **Pestaña 2 — Batería de 12 Casos de Prueba:** Permite ejecutar individualmente o en lote los 12 casos de prueba documentados para verificar en vivo escenarios con una solución, múltiples soluciones, cero coincidencias, datos incompletos y valores fuera de dominio.
- **Pestaña 3 — Explorador de la Base de Conocimientos (`.pl`):** Muestra indicadores de cumplimiento de la rúbrica y permite filtrar los 24 hechos `mascota/12` por especie y origen (raza o refugio).
- **Pestaña 4 — Arquitectura Docker y GitHub:** Documenta el `Dockerfile` y los comandos listos para copiar y desplegar.

10. Tecnologías Utilizadas

Tecnología / Herramienta	Versión / Imagen	Rol dentro del Proyecto
SWI-Prolog	`swipl:stable` (9.x)	Motor de inferencia lógica (`mascotas_kb.pl`) y servidor web HTTP/JSON nativo (`server.pl` con `thread_httpd`).
HTML5, Tailwind CSS y JS	Tailwind 3.4 / ES6	Construcción de la interfaz gráfica responsiva, validación de formularios y consumo de la API `POST /api/consultar`.
Docker y Docker Compose	OCI Container	Empaquetado reproducible del motor SWI-Prolog, la API y el frontend en un único contenedor listo para producción.
Git y GitHub	Git 2.x	Control de versiones distribuido con historial de commits semánticos, `.gitignore` y documentación `README.md`.
Render / Railway	PaaS Cloud Docker	Plataforma de despliegue en la nube con soporte nativo para contenedores Docker y URL pública HTTPS.

11. Procedimiento de Integración, Git/GitHub y Despliegue (Etapas 5, 6 y 7)

11.1. Integración Segura entre la Interfaz y Prolog (Etapa 5)

El servidor `prolog/server.pl` expone el endpoint `POST /api/consultar` utilizando `library(http/http_json)`. Para cumplir con la directriz de seguridad de la práctica (*"No deberá ejecutar directamente cadenas arbitrarias de código Prolog enviadas por los usuarios"*), el predicado `validar_y_ejecutar/8` verifica primero que cada cadena JSON corresponda a un átomo permitido mediante `valor_valido/2` antes de invocar a `obtener_recomendaciones/8`.

11.2. Administración del Repositorio en Git y GitHub (Etapa 6)

El repositorio incluido en el paquete entregable cuenta con una evolución estructurada en 5 commits descriptivos sobre la rama `main`:

```
8005533 chore(repo): configurar .gitignore y .dockerignore para el proyecto Prolog
a84d743 feat(prolog): implementar base de conocimientos mascotas_kb.pl con 61 hechos y 15 reglas
4c85bb6 feat(api): implementar servidor HTTP en SWI-Prolog (server.pl) y bateria de 12 pruebas
7b21f7b feat(frontend): desarrollar interfaz grafica web interactiva conectada al motor Prolog
ee932e1 build(docker): agregar Dockerfile con swipl:stable, render.yaml y documentacion README
```

11.3. Configuración de Docker y Publicación en Internet (Etapa 7)

El archivo `Dockerfile` parte de la imagen oficial `swipl:stable`, copia los módulos `prolog/`, `frontend/` y `tests/`, lee dinámicamente la variable de entorno `PORT` (requerida por plataformas cloud como Render y Railway) e inicia el servidor HTTP:

```
FROM swipl:stable
WORKDIR /app
COPY prolog/ ./prolog/
COPY frontend/ ./frontend/
COPY tests/ ./tests/
ENV PORT=8080
EXPOSE 8080
CMD ["swipl", "-s", "prolog/server.pl", "-g", "iniciar_servidor"]
```

Para publicarlo en Internet con URL pública accesible por el docente desde cualquier computadora o dispositivo móvil sin instalar Prolog: (1) se sube el repositorio a GitHub, (2) se vincula en **Render.com** (*New Web Service → Build and deploy from a Git repository*), (3) Render detecta automáticamente el archivo `Dockerfile` y `render.yaml`, construye la imagen con SWI-Prolog y publica el servicio en una URL HTTPS pública.

12. Pruebas y Resultados Documentados (Etapas 3 y 8)

Se diseñaron y ejecutaron **12 casos de prueba** tanto en el intérprete de SWI-Prolog (`tests/pruebas_consola.pl`) como desde la interfaz gráfica web, cubriendo todos los escenarios obligatorios de la Etapa 8:

#	Escenario / Tipo de Prueba	Entradas / Consulta en Prolog	Resultado Esperado	Resultado Obtenido y Coincidencia Web/Prolog
1	Consulta válida con solución única (Hogar con alergias en depto pequeño)	`obtener_recomendaciones(departamento_pequeno, poco, si, ninguno, principiante, moderado, cualquiera, S).`	1 solución única: `gato_azul_ruso` (hipoalergénico, baja energía).	**1 solución:** Gato Azul Ruso (89% afinidad, `compatibilidad_alta_recomendada`). Coincidencia exacta.
2	Múltiples soluciones vía Backtracking (Familia con niños pequeños y patio)	`obtener_recomendaciones(casa_con_patio, alto, no, pequenos, principiante, alto, perro, S).`	Múltiples perros con `tolerancia_ninos = alta` y experiencia principiante.	**6 soluciones:** Mestizo Adulto (99%), Cruza Poodle (99%), Bulldog Francés (91%), Pug (93%), Schnauzer Mini (93%), Golden Retriever (85%).
3	Adoptante avanzado con jardín grande y presupuesto moderado	`obtener_recomendaciones(casa_con_jardin_grande, alto, no, ninguno, avanzado, moderado, perro, S).`	Incluye perros de trabajo/alta energía; descarta los de presupuesto alto.	**9 soluciones:** Incluye Border Collie (85%), Mestizo Guardián (91%) y Beagle (89%); descarta Pastor Alemán por presupuesto.
4	Estudiante con presupuesto económico en depto pequeño	`obtener_recomendaciones(departamento_pequeno, poco, no, ninguno, principiante, economico, cualquiera, S).`	Perfiles de bajo costo y bajo requerimiento de espacio y tiempo.	**4 soluciones:** Perro Mestizo Adulto (89%), Gato Pelo Corto (89%), Gato Senior (89%) y Chihuahua (83%).
5	Restricción combinada: Alergias (`si`) + Niños pequeños (`pequenos`)	`obtener_recomendaciones(departamento_amplio, moderado, si, pequenos, principiante, moderado, cualquiera, S).`	Solo animales hipoalergénicos con alta tolerancia infantil.	**2 soluciones:** Cruza de Poodle Rescatado (99%, ideal) y Schnauzer Miniatura (90%, ideal).
6	Filtro por especie (`gato`) con niños mayores y experiencia intermedia	`obtener_recomendaciones(departamento_amplio, moderado, no, mayores, intermedio, alto, gato, S).`	Gatos de tolerancia alta o media; excluye Gato Bengalí por experiencia/tiempo.	**9 soluciones felinas:** Pareja de Gatitos (94%), Gato Ragdoll (94%), Siamés (90%), Siberiano (83%), Maine Coon (83%), etc.
7	Consulta sin coincidencias (Restricciones mutuamente excluyentes)	`obtener_recomendaciones(departamento_pequeno, poco, si, pequenos, principiante, economico, cualquiera, S).`	Lista vacía `[]` / `false` en Prolog; aviso claro en la interfaz.	**0 soluciones (`[]`):** La interfaz informa que no existen coincidencias y muestra los motivos de descarte de los 24 perfiles.
8	Prueba negativa de exclusión por seguridad infantil (Chihuahua)	`mascota(chihuahua,_,_,_,_,_,_,_,_,Tol,_,_), seguro_ninos(pequenos, Tol).`	`false` (Chihuahua tiene tolerancia `baja` y se descarta con niños pequeños).	**`false`:** Descartado con motivo `inseguro_para_ninos(tolerancia(baja))`.
9	Prueba de recursividad sobre listas (`contar_elementos/2` y `sumar_factores/2`)	`cuidados_especificos(golden_retriever, C), contar_elementos(C, N).`	`N = 3` cuidados y suma recursiva exacta de factores de bonificación.	**`N = 3`** y puntaje recursivo verificado sin desbordamiento (tope 100).
10	Diagnóstico de descarte (`diagnosticar_descarte/9` para Border Collie)	`diagnosticar_descarte(departamento_pequeno, poco, no, pequenos, principiante, economico, perro, border_collie, M).`	Lista con los 5 motivos lógicos por los que falla para ese usuario.	**5 motivos inferidos:** espacio, tiempo, seguridad infantil, experiencia avanzada y presupuesto.
11	Validación de Datos Incompletos (Parámetro vacío en solicitud)	Solicitud con `vivienda: ""` hacia `/api/consultar`.	Rechazo controlado con código `DATOS_INCOMPLETOS` sin caer el servidor.	**Validado:** Muestra mensaje solicitando completar los datos faltantes sin reiniciar la app.
12	Validación de Valores No Válidos (Átomo fuera del dominio)	Solicitud con `vivienda: "mansion_espacial"` hacia `/api/consultar`.	Rechazo controlado por `valor_valido/2` con código `VALORES_NO_VALIDOS`.	**Validado:** Bloquea el átomo desconocido y presenta alerta clara en pantalla.

13. Problemas Técnicos Encontrados y Soluciones Implementadas

- **Serialización de términos compuestos de Prolog a JSON:** Las reglas de trazabilidad generan estructuras como `regla_espacio(vivienda_usuario(casa_con_patio), requiere_minimo(departamento_pequeno))`, las cuales no son tipos primitivos JSON. Se resolvió utilizando `maplist(term_string, ReglasTerm, ReglasStr)` en `server.pl` antes de invocar a `reply_json_dict/1`.
- **Políticas CORS en navegadores modernos:** Las peticiones `POST` con `Content-Type: application/json` disparan una petición previa `OPTIONS` (preflight). Se solucionó configurando `:- set_setting(http:cors, [*]).` y atendiendo explícitamente el método `options` en el manejador `api_consultar/1`.
- **Asignación dinámica de puertos en plataformas Cloud (Render / Railway):** Los servicios PaaS inyectan el puerto mediante la variable de entorno `PORT`. Se implementó `getenv('PORT', PuertoStr)` con conversión `atom_number/2` y valor por defecto `8080` para funcionar tanto en local como en la nube sin modificar el código.

14. Conclusiones Individuales y del Equipo

Conclusión de Diego Perea Medina: El diseño e implementación de la base de conocimientos `mascotas_kb.pl` permitió comprobar en la práctica las ventajas expresivas del paradigma lógico frente al paradigma imperativo. Modelar los 24 perfiles de mascotas y sus jerarquías mediante hechos declarativos, junto con reglas de exclusión estricta para factores críticos como alergias y convivencia con niños pequeños, facilitó separar por completo el conocimiento del dominio respecto del mecanismo de control. Asimismo, el uso de unificación, recursividad sobre listas para el cálculo ponderado de afinidad y retroceso automático (backtracking) con `findall/3` demostró cómo Prolog infiere múltiples soluciones justificadas y audita las causas exactas de descarte sin necesidad de estructuras condicionales anidadas complejas.

Conclusión de Gerardo Cortés Cruz: La integración entre la interfaz gráfica web y el motor lógico mediante el servidor HTTP nativo de SWI-Prolog (`server.pl` con `thread_httpd`) y su contenerización con Docker bajo la imagen oficial `swipl:stable` evidenciaron que Prolog puede integrarse de forma limpia, segura y reproducible en arquitecturas web modernas. Validar los átomos de entrada antes de ejecutar la consulta evitó vulnerabilidades de inyección de código lógico, mientras que el flujo de trabajo con Git, GitHub y Docker permitió empaquetar tanto el frontend como el motor de inferencia en un único servicio listo para producción y accesible desde cualquier navegador en Internet.

15. Referencias y Declaración de Uso de Herramientas de IA

15.1. Referencias Bibliográficas

- Bratko, I. (2012). *Prolog Programming for Artificial Intelligence* (4th ed.). Addison-Wesley Educational Publishers.
- Wielemaker, J., Schrijvers, T., Triska, M., & Lager, T. (2012). SWI-Prolog. *Theory and Practice of Logic Programming*, 12(1-2), 67–96. Documentación oficial HTTP Server: `https://www.swi-prolog.org/pldoc/package/http.html`
- Clocksin, W. F., & Mellish, C. S. (2003). *Programming in Prolog: Using the ISO Standard* (5th ed.). Springer.

15.2. Declaración de Uso de Herramientas de Inteligencia Artificial

En cumplimiento con el apartado 4.2 y el entregable 8 de los lineamientos de la práctica integradora, se declara que se utilizaron herramientas de asistencia de Inteligencia Artificial Generativa como apoyo en la estructuración de la interfaz gráfica web con Tailwind CSS, la configuración de cabeceras CORS en la biblioteca `thread_httpd`, el diseño del contenedor `Dockerfile` para su despliegue y la maquetación tipográfica de la documentación técnica. Asimismo, el equipo comprende íntegramente el funcionamiento de todos los hechos, reglas, mecanismos de unificación, recursividad y backtracking implementados en los archivos `.pl` para su defensa y modificación en vivo.